

In this exercise, you'll work with Kiro to build a simple backend API for your chatbot using AWS Lambda and API Gateway. The backend will respond to chat messages with an acknowledgment and timestamp.

What You'll Learn

By completing this exercise, you will:

- **Build** a Lambda function to handle chat requests
- **Deploy** an API Gateway to expose your Lambda function
- **Test** your backend API independently
- **Experience** serverless development with Kiro's assistance

Step 1: Build the Backend

Give Kiro clear instructions to build your chatbot backend. Copy and paste this request:

Build a simple backend for my chatbot with these requirements:

- Create a Lambda function that handles chat messages

- Set up API Gateway to expose the Lambda function

- The Lambda function should respond to POST requests with an ACK message and timestamp

- Use "kiro-workshop" prefix for all AWS resources

- Lambda execution role should use the name "kiro-workshop-backend-LambdaExecutionRole"

- Don't modify the frontend - focus only on the backend

Please use CloudFormation to deploy everything.

Just create all the required files. Do not deploy for now.

📘 Keep It Simple

This exercise focuses on getting a working backend quickly. You can specify your preferred programming language (Python, Node.js, etc.) if you have a preference, but it's not required.

Step 2: Let Kiro Validate and Deploy

Allow Kiro to handle the validation and deployment process. When Kiro suggests running commands, ask Kiro to execute them:

Please validate the backend CloudFormation stack first, then run the deployment commands for me. Show me the results.

What to Expect

Your backend will include:

- **Lambda Function:** Processes chat messages and returns responses
- **API Gateway:** Provides an HTTP endpoint for your Lambda function
- **IAM Role:** Gives Lambda permission to write logs

✅ Let Kiro Handle It

Don't worry about the technical details right now. Focus on seeing the process work end-to-end. Kiro will handle the CloudFormation template, Lambda code, and deployment steps.

Step 3: Test the API

Once deployed, ask Kiro to test the backend API:

Please test the API endpoint by sending a sample chat message and show me the response.

Expected Response

When you send a message like "Hello, chatbot!", you should get back something like:

```
{
  "message": "Message received\: Hello, chatbot!",
  "timestamp": "2024-01-15T10\:30\:00Z"
}
```

📘 Simple Testing

The goal is to verify that your Lambda function receives messages and responds correctly. Don't worry about complex testing scenarios at this stage.

Step 4: Verify Everything Works

Ask Kiro to show you the key information about your deployment:

Please show me:

1. The API Gateway endpoint URL

2. A successful test of the API

3. The CloudWatch logs showing the Lambda function executed

This will confirm everything is working correctly.

Success Criteria

After completing this exercise, you should have:

- **✅ Working Lambda Function:** Responds to chat messages with ACK and timestamp
- **✅ API Gateway Endpoint:** Provides HTTP access to your Lambda function
- **✅ Successful Testing:** Verified the API works by sending test messages
- **✅ CloudWatch Logs:** Can see your Lambda function execution logs

✅ Backend Complete

You now have a working serverless backend! The next exercise will connect your frontend to this backend API.

Troubleshooting

If something doesn't work, try these simple fixes:

- ▶ **Deployment Fails**
- ▶ **API Not Responding**

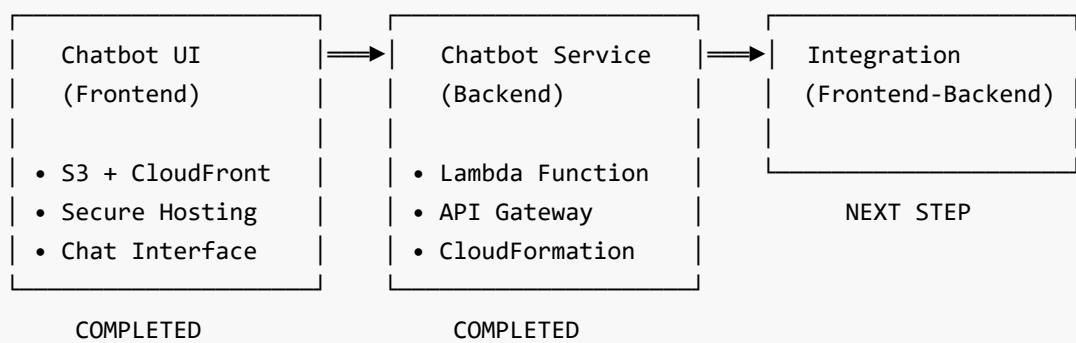
Architecture Overview

- ▶ **View Backend Architecture**

Next Steps

Now that your backend is working, the next exercise will connect your frontend chatbot UI to this backend API, creating a complete working chatbot application.

Workshop Progress



[Let's Integrate Chatbot Frontend With Backend](#)